

Mobile Robotics (MCT-454L) - Lab 3

Report

1. Objective

The objective of this lab was to transition from utilizing pre-existing ROS 2 packages to developing a custom ROS 2 workspace. This involved creating a custom Python package (my_turtle_package), writing modular publisher nodes to control the turtlesim robot, and utilizing advanced ROS 2 features such as topic remapping and closed-loop proportional control to achieve complex multi-robot behaviors.

2. File Structure and Modularity

Instead of relying on a single, monolithic script to control the robot, this lab was executed using a highly modular file structure. Inside the my_turtle_package, separate executable Python scripts were created for each specific behavior:

- my_node.py (Square pattern)
- draw_circle.py (Circular pattern)
- draw_triangle.py (Triangular pattern)
- go_to_goal.py (Interactive proportional controller)

This modular approach ensures clean code, prevents logical conflicts, and allows individual behaviors to be scaled or reassigned to different robots dynamically without altering the source code.

3. Task Implementations & Observations

3.1 Task 1: Basic Movement Patterns

Using the rclpy library, three separate nodes were developed to publish geometry_msgs/msg/Twist messages to the /turtle1/cmd_vel topic.

- The **Square** and **Triangle** nodes utilized an open-loop time-delay approach (using time.sleep()) inside a for loop to alternate between linear velocity and angular velocity (turning 90° and 120°, respectively).
- The **Circle** node utilized a continuous high-frequency timer callback to publish both linear and angular velocities simultaneously, resulting in a smooth, continuous arc.

3.2 Task 2: Multi-Turtle Control via Topic Remapping

To control three turtles simultaneously in different patterns, the /spawn service was used to instantiate turtle2 and turtle3. Rather than duplicating the Python code for each new turtle, a powerful ROS 2 CLI feature called **Topic Remapping** was utilized.

By running the commands:

- `ros2 run my_turtle_package draw_circle --ros-args --remap turtle1/cmd_vel:=turtle2/cmd_vel`
- `ros2 run my_turtle_package draw_triangle --ros-args --remap turtle1/cmd_vel:=turtle3/cmd_vel`

The existing modular nodes were dynamically instructed to redirect their velocity commands to the new turtles. This allowed all three turtles to draw their respective shapes simultaneously in the simulation without requiring any code modification.

3.3 Task 3: Interactive Closed-Loop Proportional Controller (go_to_goal)

For the final task, an interactive, closed-loop controller was developed. Unlike the open-loop shapes, this node subscribes to the `/turtle1/pose` topic to receive real-time coordinate feedback.

- **Mathematics:** The node calculates the Euclidean distance to the goal using the formula $E_d = \sqrt{(x_{\text{goal}} - x_{\text{current}})^2 + (y_{\text{goal}} - y_{\text{current}})^2}$ and the required heading using $\theta_{\text{goal}} = \text{atan2}(\Delta y, \Delta x)$.
- **Control Logic:** A Proportional (P) controller multiplies these error values by a gain constant to determine the linear and angular velocities. As the turtle approaches the target, the error decreases, causing the turtle to smoothly decelerate and stop precisely at the coordinates.
- **Interactivity:** The node was upgraded to accept standard Python `input()` before initializing the ROS 2 loop, allowing the user to dynamically enter any valid (X, Y) coordinate in the terminal for the turtle to navigate to.

4. Conclusion

This lab successfully demonstrated the creation of a custom ROS 2 package. By implementing modular design, topic remapping, and closed-loop control mathematics, the lab provided a robust foundation for developing autonomous, feedback-driven behaviors in multi-robot systems.